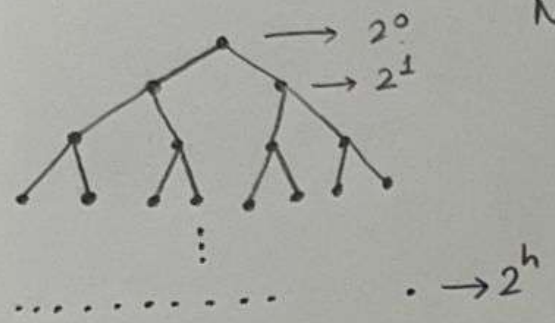
Yakup Emre Çelebi
22302267
Section: 2

Question 1 - Full Binary Tree

a) Number of leaves is 2^h and number of internal nodes is $2^h - 1$ in the full binary tree.



Number of internal nodes $= 2^0 + 2^1 + \ldots + 2^{h-1} =$
$= 2^h - 1$

For $h = 4$, number of leaves is 16, and number of internal nodes is 15.

b)

```
CountNodes (root)
    IF (root = null) THEN
        RETURN 0
    ELSE
        RETURN CountNodes(root->right) +
               Count Nodes(root -> left) + 1
    ENDIF
```

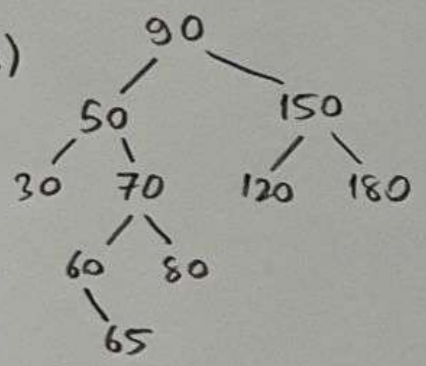
Running Time is $O(2^h)$

Yakup Emre Gelebi
22302267
Section: 2

Question 2- Searching a Binary Search Tree

```
SearchBST(root, key)
    currentNode ← root
    WHILE (currentNode != null) DO
        IF (key = currentNode→key) THEN
            RETURN currentNode
        ELSIF (key < currentNode→key) THEN
            currentNode ← currentNode→left
        ELSE
            currentNode ← currentNode→right
        ENDIF
    ENDWHILE
    RETURN null
```

a)



Sequence of keys visited when searching for the key 65 : 90, 50, 70, 60, 65

b) Number of comparisons is 5 since we scan the loop 5 times.

Yakup Emre Gelebi
22302267
Section: 2

Question 3 - Height of a Binary Search Tree

b) Returned value in height() function is 5, it is between the minimum and the minimum.
Becouse, we have 10 nodes. The minimum height is when the tree is full balanced, and in that case number of nodes is $2^h - 1$. Thus, minimum height is 4, since if it was 3, then $2^3 - 1 = 7 \rightarrow$ Contradiction.
Also, maximum height is when the tree is like a stick, which is 10. So, our height is between these numbers.

c) The path makes tree's height 5 is 40, 20, 30, 35, 32.
So, deleting one of these makes height 4.
If I wanted to drop the height immediately, I would choose node 32. Becouse, it has no child and cost is minimum since it is $O(1)$.

Yakup Emre Gelebi
22302287
Section: 2

Question 4 - TreeSort Analysis

a) Running Time : $O(n^2)$

Since the tree is like a sorted list, inserting first node is 1 unit time, inserting second node is 2 unit time, and inserting nth node takes N unit time. So, $1+2+\ldots+n = \frac{n^2+n}{2}$. So, this is $O(n^2)$. And Inorder Traversal is $O(n)$, since every node traveled once. So, total is $O(n^2)$

b) Running Time : $O(n.\log n)$

For insertion, if the tree is balanced height is $\approx \log n$. So every node takes $\log n$. So, total $O(n.\log n)$

Inorder Traversal is $O(n)$, same since node number is n. So, total is $O(n.\log n)$

Yakup Emre Gelebi
22302267
Section:2

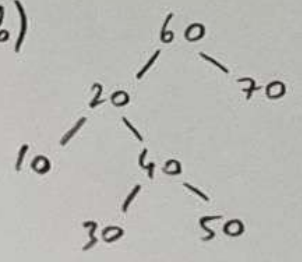
Bonus Question - Saving and Restoring Binary Search Trees

a)

```
saveBST(root)
  IF(root=null) THEN
     RETURN
  ENDIF
  Add root→key to list
  saveBST(root→left)
  saveBST(root→right)

restoreBST(file)
  root ← null
  WHILE (file.length >0) DO
     key ← file.next
     root ← insert(root, key)
  ENDWHILE
  RETURN root
```

b)



Sequence of keys written by saveBST:

60, 20, 10, 40, 30, 50, 70

Since first one is 60, root is again 60. 20 is still left side of 60 and 70 is right side of 60 and continues like that. So, this tree has the same shape with the first tree

Bonus Question

c)

```
buildMinHeightBST(sortedList)
    IF (sortedList.length = 0) THEN
        RETURN null
    ENDIF

    midInd ← sortedList.length /2
    newNode ← TreeNode (sortedList [mid])
    leftPartOfTheList ← sortedList (0, midInd -1)
    newNode→left ← buildMinHeightBST (leftPartOfTheList)
    rightPartOfTheList ← sortedList (mid+1, sortedList.last)
    newNode→right ← buildMinHeightBST (rightPartOfTheList)
    RETURN newNode
```

Height is 3. And it is less than original tree from part b since it is fully balanced.